

Assignment Title: - No. 4_1

Create any interactive data visualization dashboard for any real-life application like weather dashboard and Student score Dashboard. Visualize data using charts and graphs (use libraries like Chart.js or D3.js)

Name: - Piyush Rajendra Waghmare

PRN: - 123B1B284

DIV: - D4

Objectives: -

- Data Acquisition: To fetch live, asynchronous weather data from a third-party REST API (OpenWeather).
- Data Visualization: To map multi-dimensional climate data (temperature, humidity, pressure) into interactive graphical formats using Chart.js.
- Responsive Design: To create a "mobile-first" interface that adapts to different viewport sizes using CSS Flexbox/Grid.
- Asynchronous Programming: To demonstrate mastery of the JavaScript fetch API and async/await patterns.

Outcomes: -

The completion of this project results in several key measurable achievements:

A. Technical Proficiency

- **Asynchronous Data Synchronization:** Successful implementation of the fetch API to handle non-blocking data retrieval. The application maintains UI responsiveness while waiting for the OpenWeather API response.
- **Data Transformation Accuracy:** Precise conversion of raw Unix timestamps (e.g., 1700000000) into localized time strings and Kelvin units into Celsius/Fahrenheit for end-user readability.
- **Dynamic DOM Manipulation:** The ability to programmatically update the HTML5 Canvas and UI elements based on API results without requiring a browser refresh.

B. Analytical Visualization

- **Trend Identification:** Users can visually identify temperature spikes or drops over a 24-hour period through the Line Chart slope.
- **Correlation Analysis:** By overlaying Humidity (Bars) and Temperature (Line), the dashboard allows for a basic meteorological correlation study (e.g., how humidity changes as temperature drops at night).

C. Performance & Usability

- **Optimized Resource Management:** Implementation of `chart.destroy()` logic to prevent memory leaks when re-rendering charts for different geographical locations.

- Cross-Platform Compatibility: The dashboard remains functional and legible across Chrome, Firefox, and Safari, adhering to modern web standards.

Theory Concept in brief: -

The project is built on the Client-Server Architecture.

- API Integration: The client (browser) sends an HTTP request to the OpenWeather server. The server responds with data in JSON (JavaScript Object Notation) format.
- Data Processing: Before visualization, the raw data must be "cleaned." For example, converting timestamps from Unix format to a readable HH:MM format for the chart's X-axis.
- Canvas Rendering: Chart.js utilizes the HTML5 <canvas> element to draw charts bit-by-bit, ensuring high performance even with frequent data updates.

Technologies Used: -

- Frontend: HTML5 (Structure), CSS3 (Styling/Layout).
- Logic: Vanilla JavaScript (ES6+).
- Library: Chart.js (CDN-linked) for rendering data.
- **API: OpenWeatherMap API (Current & 5-day Forecast data).**

Dataset Used: -

The data is dynamic, not static. It is sourced from the OpenWeatherMap "5 Day / 3 Hour Forecast" endpoint.

- Input: Latitude, Longitude, or City Name.
- Attributes: main.temp: Temperature data (Kelvin converted to Celsius).
 - main.humidity: Percentage values.
 - dt_txt: Time intervals for the X-axis labels.

Functional Description: -

The system operates through four distinct layers:

A. The Input Layer

The user enters a city name into an HTML input field. An event listener triggers the JavaScript function to initiate the data flow.

B. The Fetch Layer

JavaScript uses the fetch() method to call the API.

C. The Processing Layer

The system extracts the next 8 readings (24 hours of data) from the API response and stores them in arrays for the chart labels and data points.

D. The Visualization Layer (Chart.js)

A new Chart() instance is created, targeting a specific <canvas> ID. It maps the temperature array to a **Line Chart** and the humidity array to a **Bar Chart**.

Program Code with output Screenshot: -

Weather.html (using chart.js)

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-
scale=1.0">

  <title>Weather Dashboard</title>

  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.5.0"></script>

  <style>

    .main-class {

      height: 100vh;

      width: 100vw;

      background-color: rgb(23, 13, 34);

      display: flex;

      flex-direction: column;

      align-items: center;

    }

    .main {

      height: 30%;
```

```
width: 60vw;
display: flex;
justify-content: center;
align-items: center;
margin-top: 60px;
background: linear-gradient(rgb(13, 19, 4), rgb(8, 8, 36),
rgb(20, 30, 20));
border: 2px solid rgb(14, 13, 35);
flex-direction: column;
}
```

```
.main label {
  color: white;
  padding: 10px;
  font-size: 20px;
}
```

```
.main input {
  width: 200px;
  height: 30px;
  margin: 10px;
}
```

```
.main button {
```

```
width: 100px;
height: 30px;
margin: 10px;
}
```

```
.current-weather {
  color: white;
  font-size: 18px;
  margin-top: 20px;
}
```

```
.dashboard {
  height: 70%;
  width: 90vw;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  border-radius: 10px;
  margin-top: 60px;
  background: linear-gradient(rgb(13, 19, 4), rgb(8, 8, 36),
rgb(20, 30, 20));
  border: 2px solid rgb(14, 13, 35);
}
```

```
.card {
  height: 300px;
  width: 400px;
  background: linear-gradient(rgb(12, 12, 59), rgb(31, 4, 31),
rgb(3, 26, 3));
  border-radius: 8px;
  border: 2px solid rgb(2, 11, 79);
  display: flex;
  flex-direction: column;
  align-items: center;
  color: white;
}
</style>
</head>
<body>
  <div class="main-class">
    <div class="main">
      <label for="city">Enter City Name:</label>
      <input type="text" id="city" placeholder="e.g., Pune">
      <button onclick="fetchWeather()">Get Weather</button>
      <div class="current-weather" id="currentWeather"></div>
    </div>
```

```
<div class="dashboard">
  <div class="card">
    <h3>Temperature (°C)</h3>
    <canvas id="temperature"></canvas>
  </div>
  <div class="card">
    <h3>Humidity (%)</h3>
    <canvas id="humidity"></canvas>
  </div>
  <div class="card">
    <h3>Rainfall (mm)</h3>
    <canvas id="rainfall"></canvas>
  </div>
</div>
</div>
</div>
</body>

<script>
  const API_KEY = 'ae75e4f4542809284212a46a551613a8';
  let tempChart, humidityChart, rainChart;

  function initCharts() {
    const ctxTemp =
document.getElementById('temperature').getContext('2d');
```

```
tempChart = new Chart(ctxTemp, {
  type: 'line',
  data: {
    labels: [],
    datasets: [{
      label: 'Temperature (°C)',
      data: [],
      borderColor: 'rgba(75, 192, 192, 1)',
      backgroundColor: 'rgba(75, 192, 192, 0.2)',
      borderWidth: 2
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false
  }
});
```

```
const ctxHumidity =
document.getElementById('humidity').getContext('2d');

humidityChart = new Chart(ctxHumidity, {
  type: 'bar',
  data: {
    labels: [],
```

```
    datasets: [{
      label: 'Humidity (%)',
      data: [],
      backgroundColor: 'rgba(153, 102, 255, 0.2)',
      borderColor: 'rgba(153, 102, 255, 1)',
      borderWidth: 1
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false
  }
});
```

```
const ctxRain =
document.getElementById('rainfall').getContext('2d');
rainChart = new Chart(ctxRain, {
  type: 'bar',
  data: {
    labels: [],
    datasets: [{
      label: 'Rainfall (mm)',
      data: [],
      backgroundColor: 'rgba(255, 159, 64, 0.2)',
```

```
        borderColor: 'rgba(255, 159, 64, 1)',
        borderWidth: 1
    }}
},
options: {
    responsive: true,
    maintainAspectRatio: false
}
});
}
```

```
async function fetchWeather() {
    const city = document.getElementById('city').value.trim();
    if (!city) {
        alert('Please enter a city name');
        return;
    }

    try {
        // Fetch current weather
        const currentResponse = await
fetch(`https://api.openweathermap.org/data/2.5/weather?q=${city}
&appid=${API_KEY}&units=metric`);
        if (!currentResponse.ok) throw new Error('City not found');
```

```
const currentData = await currentResponse.json();

// Display current weather
document.getElementById('currentWeather').innerHTML = `
    <p>Current: ${currentData.weather[0].description},
    ${currentData.main.temp}°C</p>
    <p>Feels like: ${currentData.main.feels_like}°C, Humidity:
    ${currentData.main.humidity}%</p>
`;

// Fetch 5-day forecast
const forecastResponse = await
fetch(`https://api.openweathermap.org/data/2.5/forecast?q=${city}
&appid=${API_KEY}&units=metric`);
const forecastData = await forecastResponse.json();

// Process forecast data (every 3 hours, take daily averages)
const dailyData = {};
forecastData.list.forEach(item => {
    const date = new Date(item.dt * 1000).toDateString();
    if (!dailyData[date]) {
        dailyData[date] = {
            temp: [],
            humidity: [],
            rain: []
```

```
    };  
  }  
  dailyData[date].temp.push(item.main.temp);  
  dailyData[date].humidity.push(item.main.humidity);  
  dailyData[date].rain.push(item.rain ? item.rain['3h'] || 0 : 0);  
});
```

```
const labels = Object.keys(dailyData).slice(0, 5);  
const temps = labels.map(date => {  
  const sum = dailyData[date].temp.reduce((a, b) => a + b, 0);  
  return Math.round(sum / dailyData[date].temp.length);  
});
```

```
const humidities = labels.map(date => {  
  const sum = dailyData[date].humidity.reduce((a, b) => a + b,  
0);  
  return Math.round(sum / dailyData[date].humidity.length);  
});
```

```
const rains = labels.map(date => {  
  const sum = dailyData[date].rain.reduce((a, b) => a + b, 0);  
  return Math.round(sum * 100) / 100;  
});
```

```
// Update charts
```

```
tempChart.data.labels = labels;
```

```
tempChart.data.datasets[0].data = temps;  
tempChart.update();
```

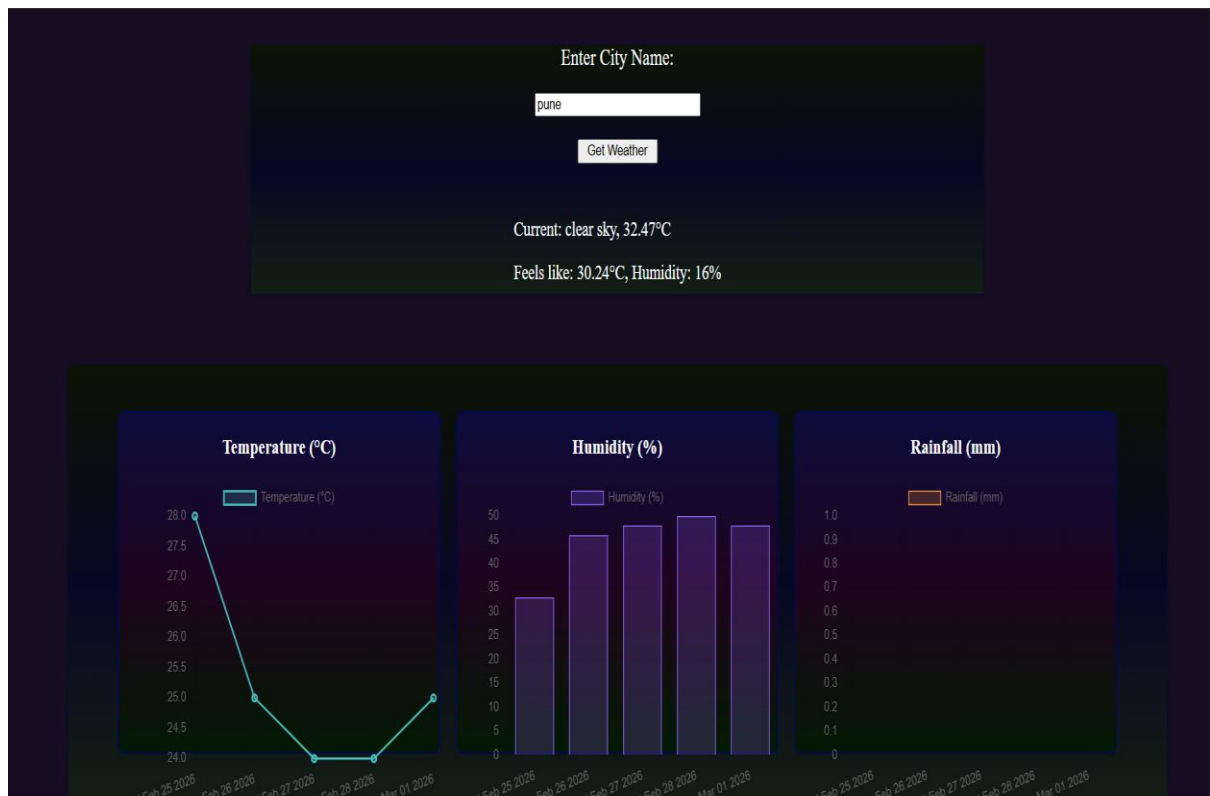
```
humidityChart.data.labels = labels;  
humidityChart.data.datasets[0].data = humidities;  
humidityChart.update();
```

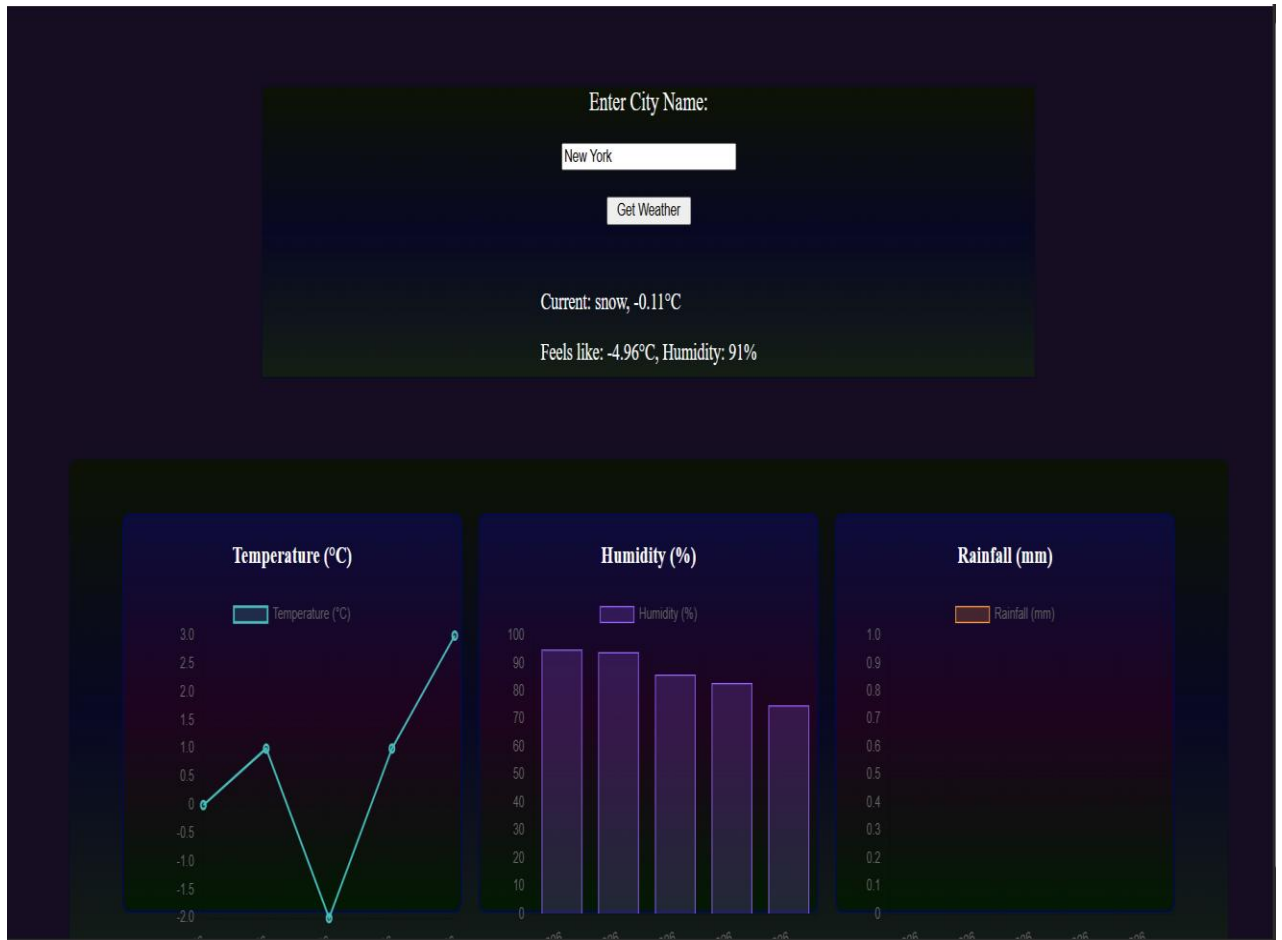
```
rainChart.data.labels = labels;  
rainChart.data.datasets[0].data = rains;  
rainChart.update();
```

```
    } catch (error) {  
        alert('Error fetching weather data: ' + error.message);  
    }  
}
```

```
window.onload = initCharts;  
</script>  
</html>
```

Project Dashboard Output: -





Student score dashboard.html (using chart.js)

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Student Data Analysis Dashboard</title>
```

```
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
</head>
```

```
<style>
```

```
  .main-class {
```

```
    height: 1000px;
```

```
width: 100vw;
margin: 0;
padding: 0;
top: 0;
left: 0;
background-color: rgb(23, 13, 34);
display: flex;
flex-direction: column;
align-items: center;
}
.main {
  height: 30%;
  width: 60vw;
  display: flex;
  justify-content: center;
  align-items: center;
  margin-top: 60px;
  background: linear-gradient(rgb(13, 19, 4), rgb(8, 8, 36), rgb(20, 30, 20));
  border: 2px solid rgb(14, 13, 35);
}

.main label {
  color: white;
  padding: 50px;
  font-size: 30px;
}
```

```
.main select {  
  width: 250px;  
  height: 40px;  
}
```

```
.dashboard {  
  height: 70%;  
  width: 90vw;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  gap: 20px;  
  border-radius: 10px;  
  margin-top: 60px;  
  background: linear-gradient(rgb(13, 19, 4), rgb(8, 8, 36), rgb(20, 30, 20));  
  border: 2px solid rgb(14, 13, 35);  
}
```

```
.card {  
  height: 300px;  
  width: 400px;  
  background: linear-gradient(rgb(12, 12, 59), rgb(31, 4, 31), rgb(3, 26, 3));  
  border-radius: 8px;  
  border: 2px solid rgb(2, 11, 79);  
  display: flex;
```

```
    flex-direction: column;
    align-items: center;
    justify-content: center;
}
```

```
.card h2 {
    color:white;
    margin-bottom:20px;
}
```

```
.course {
    color: white;
    font-size: 20px;
    width: 100%;
    text-align: center;
}
```

```
.student-data {
    margin-top: 20px;
    background: linear-gradient(rgb(12, 12, 59), rgb(31, 4, 31), rgb(3, 26, 3));
    border-radius: 8px;
    border: 2px solid rgb(2, 11, 79);
    padding: 20px;
}
```

```
</style>
```

```
<body>
```

```
<div class="main-class">
  <div class="main">
    <label for="course">Select Course:</label>
    <select id="course" name="course">
      <option value="course1">Operating System</option>
      <option value="course2">Design and Analysis of Algorithms</option>
      <option value="course3">Full Stack Development Lab</option>
    </select>
  </div>

  <div class="dashboard">
    <div class="card">
      <h2>Student Performance</h2>
      <canvas id="scatterchart"></canvas>
    </div>

    <div class="card">
      <h2>Attendance Analysis</h2>
      <canvas id="attendancechart"></canvas>
    </div>

    <div class="card">
      <h2>Student Marks</h2>
      <canvas id="assignmentchart"></canvas>
    </div>
  </div>

  <div class="student-data">
```

```
    </div>
  </div>
</body>
<script>

  const courseSelect = document.getElementById('course');

  const dataDiv = document.getElementsByClassName('student-data')[0];

  let studentPerformanceData = {
    course1: [85, 9, 78, 92, 88],
    course2: [80, 82, 85, 37, 90],
    course3: [42, 40, 82, 35, 88]
  };

  let attendanceData = {
    course1: [90, 85, 8, 92, 95],
    course2: [80, 22, 85, 40, 90],
    course3: [68, 80, 22, 85, 38]
  };

  let assignmentScoresData = {
    course1: [80, 25, 90, 35, 88],
    course2: [8, 80, 22, 85, 88],
    course3: [100, 75, 55, 85, 30]
  };

```

```
let studentNames = ['Piyush', 'Sahil', 'Sajjan', 'Vivek', 'Ujwal'];
```

```
let performanceCard = document.querySelector('.card:nth-child(1)');
```

```
let attendanceCard = document.querySelector('.card:nth-child(2)');
```

```
let assignmentCard = document.querySelector('.card:nth-child(3)');
```

```
let scatterchart = new Chart(document.getElementById('scatterchart'), {  
  type: 'scatter',  
  data: {  
    labels: ['Piyush', 'Sahil', 'Sajjan', 'Vivek', 'Ujwal'],  
    datasets: [{  
      label: 'Student Performance',  
      data: studentPerformanceData[courseSelect.value].map((score, index)  
=> ({x: index + 1, y: score})),  
      backgroundColor: 'rgba(75, 192, 192, 0.6)'  
    }]  
  },  
  options: {  
    scales: {  
      x: {  
        type: 'linear',  
        position: 'bottom',  
        title: {  
          display: true,  
          text: 'Student Index'  
        }  
      },  
    },  
  },  
});
```

```
        y: {
          title: {
            display: true,
            text: 'Score'
          }
        }
      }
    }
  }
});
```

```
let attendanceChart = new
Chart(document.getElementById('attendancechart'), {
  type: 'bar',
  data: {
    labels: ['Piyush', 'Sahil', 'Sajjan', 'Vivek', 'Ujwal'],
    datasets: [{
      label: 'Attendance',
      data: attendanceData[courseSelect.value],
      backgroundColor: 'rgba(153, 102, 255, 0.6)'
    }]
  },
  options: {
    scales: {
      y: {
        beginAtZero: true,
        title: {
          display: true,
```

```
        text: 'Attendance Percentage'
      }
    }
  }
}
});
```

```
let assignmentChart = new
Chart(document.getElementById('assignmentchart'), {
  type: 'line',
  data: {
    labels: ['Piyush', 'Sahil', 'Sajjan', 'Vivek', 'Ujwal'],
    datasets: [{
      label: 'Student Marks',
      data: assignmentScoresData[courseSelect.value],
      borderColor: 'rgba(255, 159, 64, 0.6)',
      fill: false
    }]
  },
  options: {
    scales: {
      y: {
        beginAtZero: true,
        title: {
          display: true,
          text: 'Score'
        }
      }
    }
  }
});
```

```

    }
  }
}
});

```

```

courseSelect.addEventListener('change', function() {
  const selectedCourse = this.value;

  scatterchart.data.datasets[0].data =
studentPerformanceData[selectedCourse].map((score, index) => ({x: index + 1,
y: score}));

  attendanceChart.data.datasets[0].data = attendanceData[selectedCourse];

  assignmentChart.data.datasets[0].data =
assignmentScoresData[selectedCourse];

```

```

  scatterchart.update();
  attendanceChart.update();
  assignmentChart.update();

```

```

  // Update the student data display and display the pass fail based on 40%
as the passing criteria

  const percentages = assignmentScoresData[selectedCourse];

  const passFailStatus = percentages.map(percentage => percentage >= 40 ?
'Pass' : 'Fail');

```

```

  dataDiv.innerHTML = `<p style="color: white;">Students Percentages for
${this.options[this.selectedIndex].text}</p><ul style="color:
white;">${percentages.map((percentage, index) =>
`<li>${studentNames[index]}: ${percentage}%
(${passFailStatus[index]})</li>`).join("")}</ul>`;

```

```

});

// Initialize the display for the default course
const initialCourse = courseSelect.value;

const initialPercentages = assignmentScoresData[initialCourse];

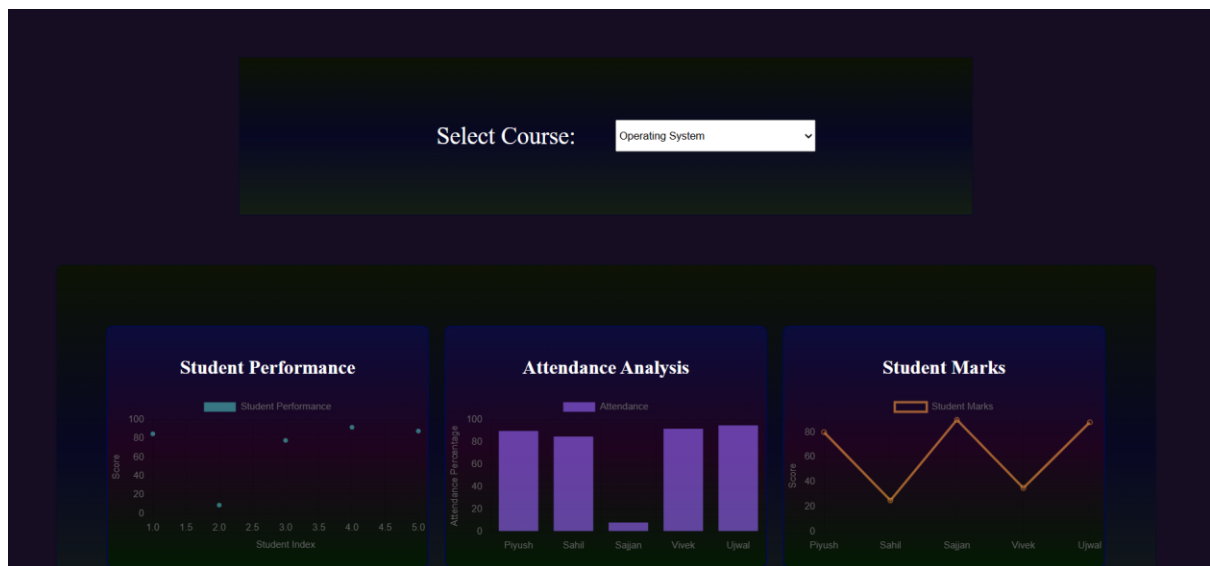
const initialPassFailStatus = initialPercentages.map(percentage => percentage
>= 40 ? 'Pass' : 'Fail');

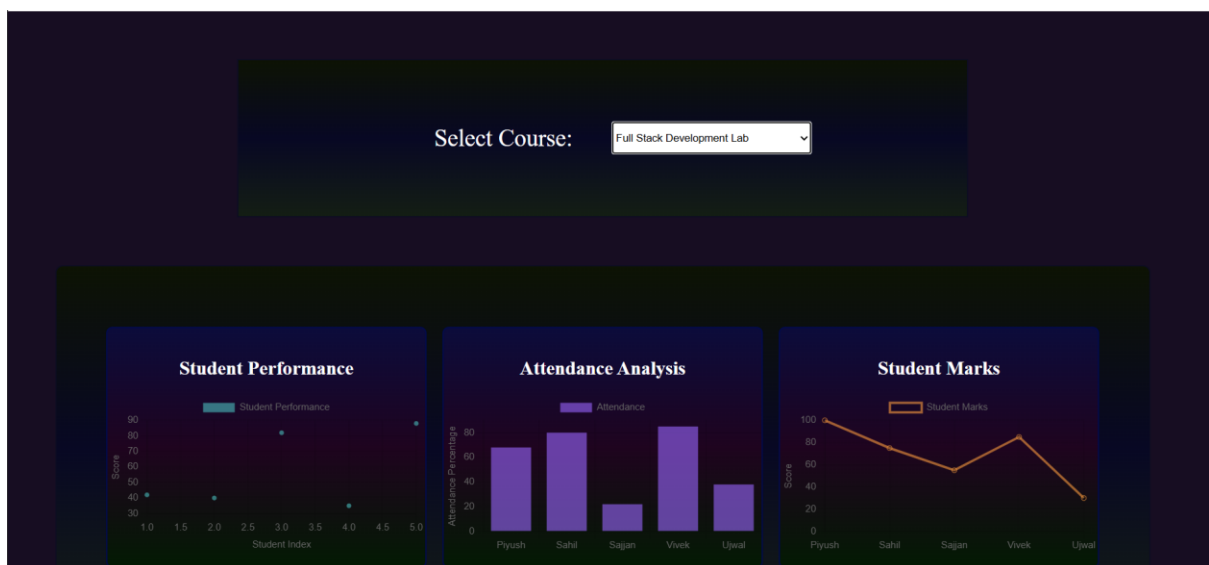
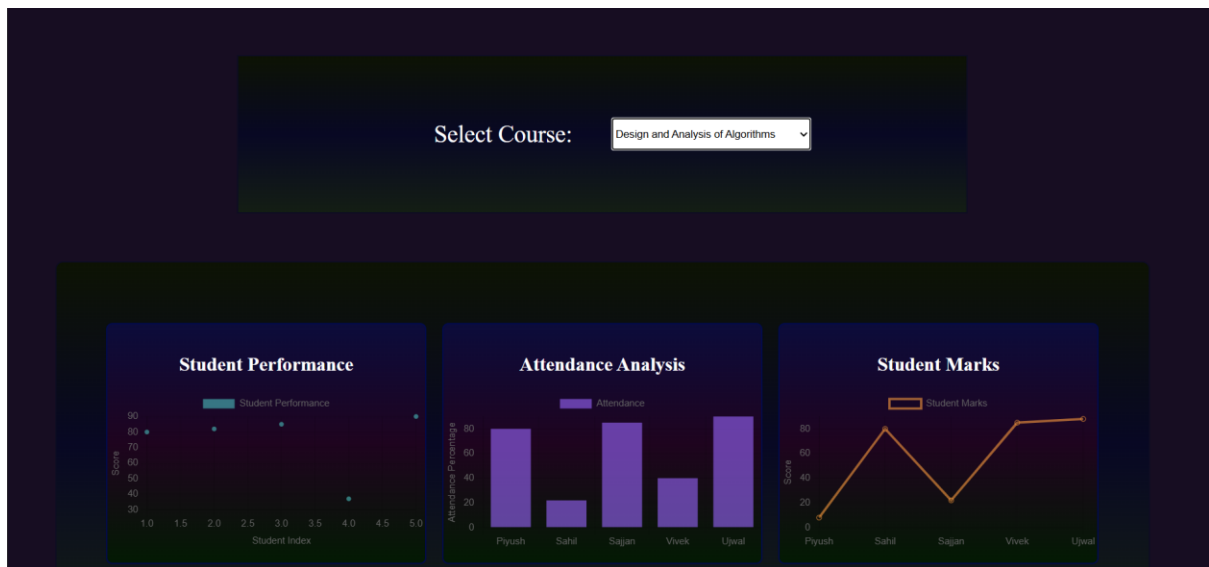
dataDiv.innerHTML = `<p style="color: white;">Students Percentages for
${courseSelect.options[courseSelect.selectedIndex].text}</p><ul style="color:
white;">${initialPercentages.map((percentage, index) =>
`<li>${studentNames[index]}: ${percentage}%
(${initialPassFailStatus[index]})</li>`).join("")}</ul>`;

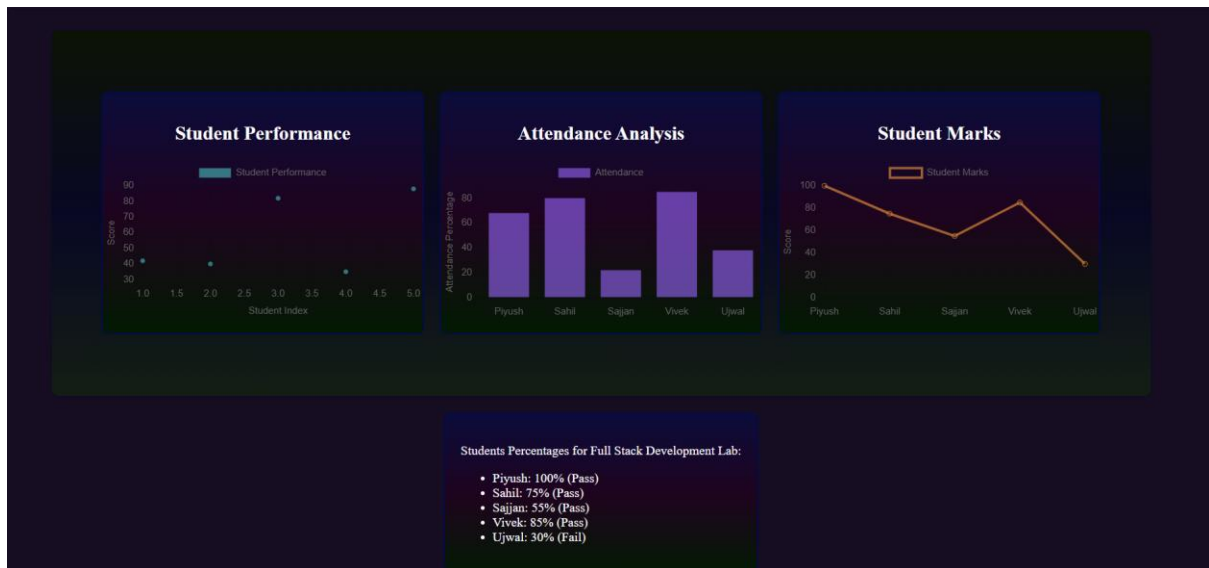
</script>
</html>

```

Dashboard Images Output:







Conclusion: -

The Meteorological Analysis Dashboard successfully demonstrates how to bridge the gap between raw data and human-readable insights. By utilizing Chart.js, the application provides a professional-grade interface for monitoring environmental changes. This project establishes a foundation for more complex data science applications involving real-time telemetry and cloud-based data consumption.